Mini Project 1.1: Simulating and Measuring Real-Time Task Timing

1. Setup

The job function (numeric_job in common/rts_common.py) burns a calibrated amount of CPU. Its work size is drawn from a normal distribution (mean 14 ms, standard deviation 4 ms, clipped to 4–30 ms), and 10% of calls take a "deep" path with an extra half-size pass. This imitates the execution-time variability of Section 1.7. The periodic loop uses T = 50 ms and D = 0.8·T = 40 ms and releases 150 jobs. It sleeps until the absolute instant r0 + k·T, recomputed from r0 each iteration, so it does not drift. Release, completion and deadline instants are stored for every job. All numbers below come from mini_project_1_1.py and vary between machines and runs; this is a simulation for illustrating concepts, not a certifiable measurement.

2. Summary (Step 9)

Quantity	Unloaded	With background busy thread
Mean / max response time R (ms)	16.54 / 49.41	26.00 / 79.19
Release jitter: std / max |.| (ms)	0.04 / 0.55	3.00 / 34.35
Completion jitter: std / max |.| (ms)	6.14 / 32.86	11.51 / 52.70
Deadline-hit fraction (R ≤ D)	98.7%	90.0%
Mean utility: hard (penalty −10) / firm / soft	0.85 / 0.99 / 0.99	-0.10 / 0.90 / 0.95

Execution time in isolation (300 calls, measure_isolated): mean 15.96 ms (32% of T), standard deviation 5.23 ms, 95th percentile 24.89 ms and maximum 40.33 ms (81% of T, just above D). The maximum is a **measured, not proven** WCET estimate: no test campaign guarantees that the longest path and the worst hardware state were exercised, whereas a statically derived bound (Section 1.7) covers all paths. Task 2 below confirms the risk, since adversarial inputs exceed this maximum.

Timing behavior. Response times cluster near the mean but have a long tail (Figure 2). Release jitter is tiny when unloaded, because the sleep-based loop wakes almost on time. Completion jitter is far larger, because it inherits the execution-time variation. Figure 3 shows that the tail decides schedulability: if D were set equal to the mean response time the hit fraction would be only 58.7%, and at the 95th percentile it is 94.7%.

Which cost function? A periodic numerical update like this one (a filter or sensor-processing step in an industrial or automotive controller, Section 1.1) would most realistically face a **firm** deadline: a result that arrives after D describes a stale state and is discarded, but one miss is not catastrophic. A **hard** classification would need a proven WCET bound, which this platform cannot give, given a miss rate of 1.3% even unloaded. A **soft** classification suits multimedia-style tasks where a late frame is still partly useful.

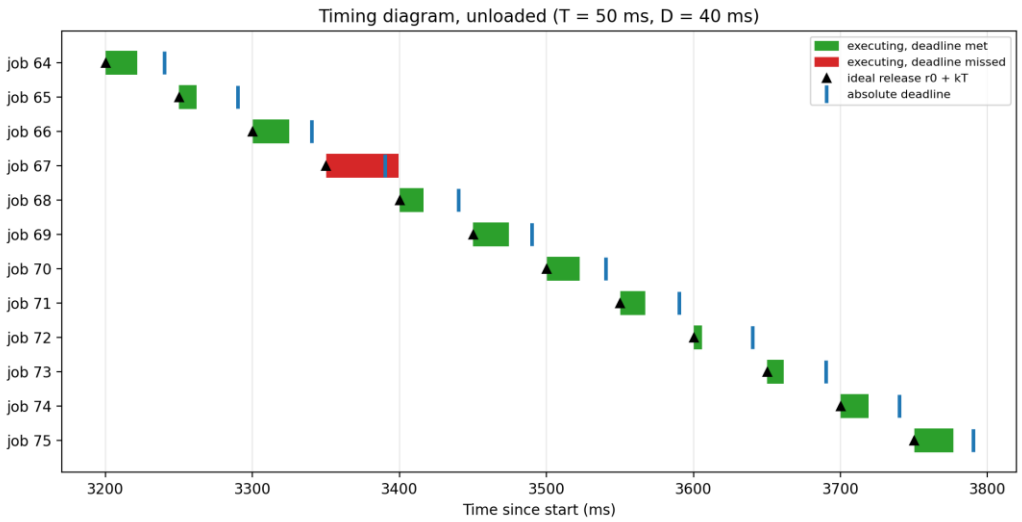


Figure 1: Timing diagram of 12 unloaded jobs (▲ ideal release, | absolute deadline, red bars = deadline missed).

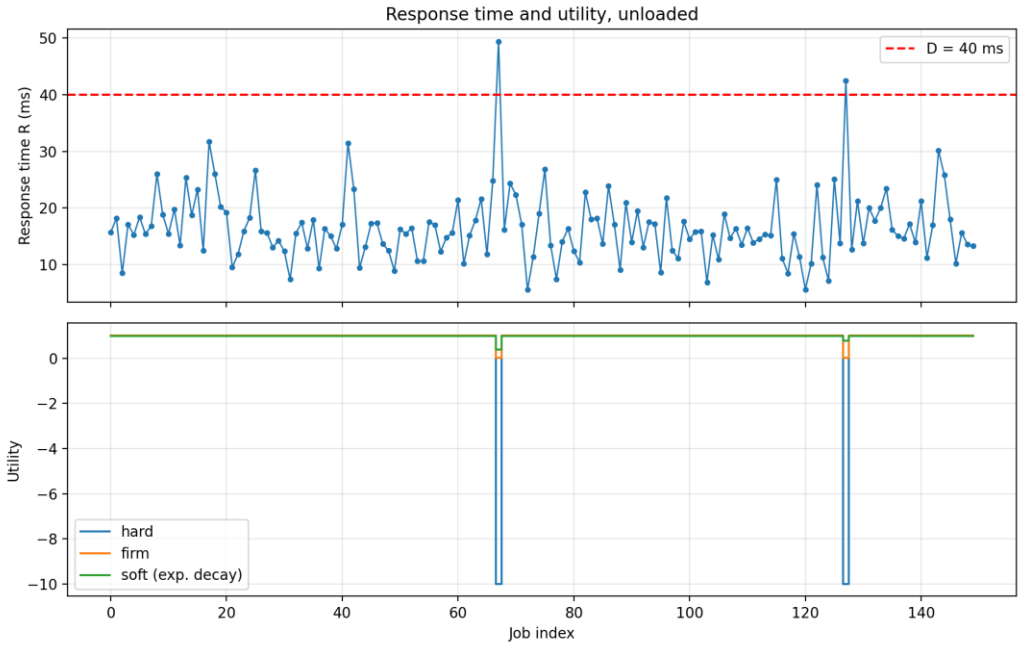


Figure 2: Response time with the deadline line (top) and hard, firm and soft utilities from the same response times (bottom).

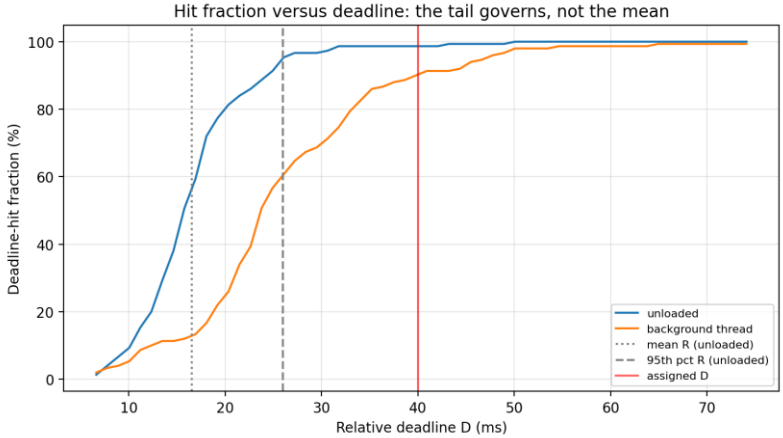


Figure 3: Deadline-hit fraction versus D, computed from the recorded response times.

3. Lab tasks

Task 1: background busy thread

A thread doing continuous arithmetic (LoadThread(1.0)) raised the release-jitter standard deviation from 0.04 ms to 3.00 ms and its maximum from 0.55 ms to 34.35 ms. The mean response time rose from 16.54 ms to 26.00 ms and the hit fraction fell from 98.7% to 90.0%. In CPython both threads share one interpreter lock, so the timer thread waits for the busy thread before it can run and the job's own compute is time-sliced. This reproduces, in mild form, the OS interference of Section 1.6. The hard utility becomes negative (-0.10), while the firm and soft utilities stay high (0.90 and 0.95), although the timing is the same. See Figures 4 and 5.

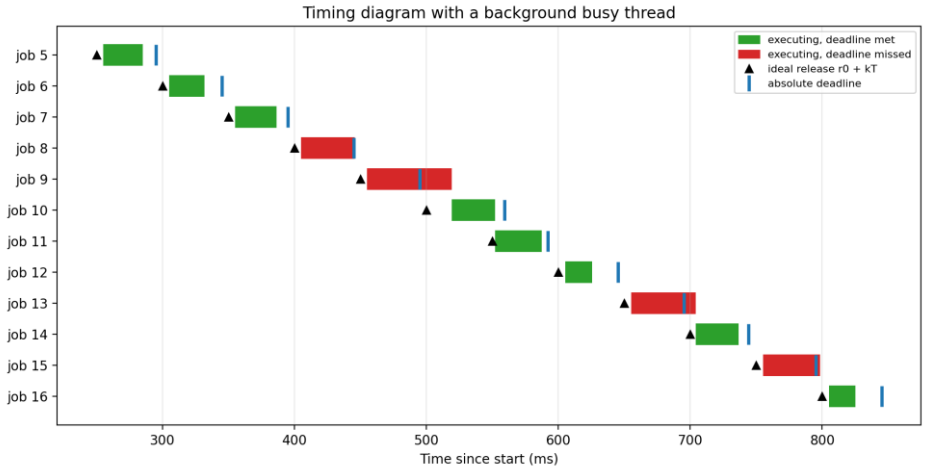


Figure 4: Timing diagram with the background thread. More jobs miss their deadline.



Figure 5: Response time and utilities with the background thread.

Task 2: adversarial inputs

Using the maximum work size with the deep path (150 calls), the mean execution time was 46.40 ms and the maximum 71.11 ms, against an ordinary-input maximum of 40.33 ms (1.76 times larger). Almost every adversarial call exceeded the ordinary maximum (Figure 6), and all of them exceeded D. Neither number substitutes for a sound static bound (Section 1.7):

the adversarial inputs were chosen by reading the code, and neither campaign can show that no longer path or worse cache or pipeline state exists.

Task 3: soft-deadline decay rate

With decay constants of 2 ms, 10 ms and 50 ms, the mean soft utility of the same loaded run is 0.913, 0.950 and 0.984 (Figure 7). The timing is identical, so the whole spread comes from the assumption. The decay rate must therefore be justified by the application, for example how quickly a late result loses value for the user or for control stability, not chosen for convenience.

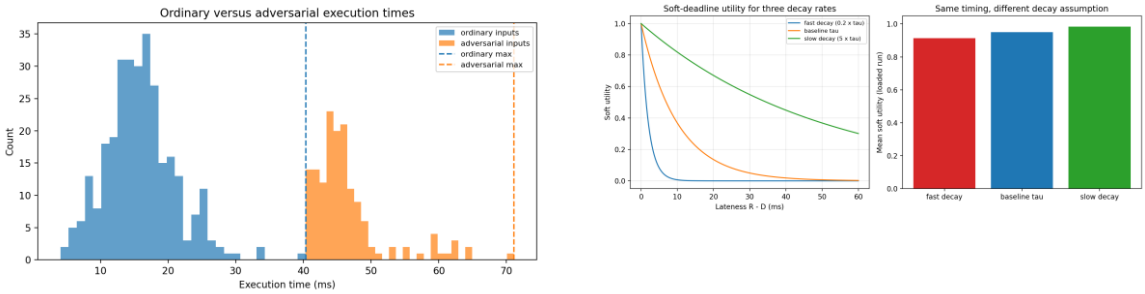


Figure 6 (left): ordinary versus adversarial execution times. Figure 7 (right): soft-deadline utility for three decay rates.

Task 4: varying the period (D = 0.8·T)

T (ms)	D (ms)	Release jitter std (ms)	Std as % of T	Max (ms)	Max as % of T	Hit fraction
30	24	0.61	2.03%	4.74	15.80%	84.0%
50	40	0.03	0.06%	0.31	0.61%	100.0%
100	80	0.29	0.29%	3.04	3.04%	100.0%
200	160	0.03	0.02%	0.29	0.14%	100.0%

Absolute release jitter stays between a fraction of a millisecond and a few milliseconds, since the timer and scheduler cause it rather than the period, so as a percentage of the period it generally shrinks as T grows. The T = 100 ms maximum comes from a single OS spike, so the trend is not perfectly monotone in one short run. At T = 30 ms the deadline (24 ms) is shorter than many job durations, jobs overrun into the next period, and release jitter reaches 15.8% of the period with only 84.0% of deadlines met. Jitter matters even when every deadline is met, because periodic activities such as sampling or filtering assume a constant interval, and a jitter of a few percent of T already distorts that assumption.

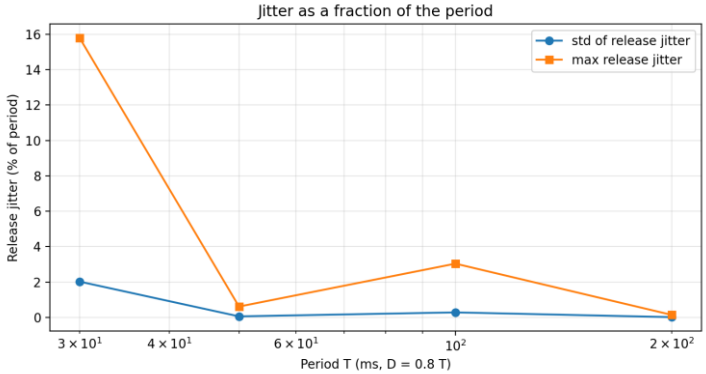


Figure 8: Release jitter as a percentage of the period.